

TITAN PORTAL: CUSTOMER-FACING SERVICE APP

Customer Service Booking, Status Tracking, Invoicing, Support

System: Titan Portal (Part of TitanBOS - Titan Business Operating System) AI Core: Titan Zero

(Cognitive decision engine) Timeline: 16 weeks (4 months) Team: 2 developers + 1 designer + 1

QA Total LOC: 18,000-22,000 lines added Platforms: iOS + Android (Flutter) + Web (PWA) Target

Users: Customers, clients, end users of service businesses

PHASE 0: FOUNDATION (Weeks 1-2)

Security + Customer Portal Framework

0.1 Security (2 hours)

[Same as before: API keys, token encryption, pinning, audit logging]

0.2 Customer Portal Framework (20 hours)

// Customer user model

```
class Customer {
```

```
  final String id;
```

```
  final String name;
```

```
  final String email;
```

```
  final String phone;
```

```
  final String address;
```

```
  final String? companyName;
```

```
  final String? abn;
```

// Preferences

```
  final String? preferredContact; // email, sms, phone, push
```

```
  final String? language;
```

```
  final bool receiveNotifications;
```

```
  final bool receiveNewsletters;
```

// Account

```
  final DateTime createdAt;
```

```
  final DateTime? lastLoginDate;
```

```
  final String accountStatus; // active, suspended, closed
```

// Payment

```
  final String? defaultPaymentMethodId;
```

```
  final List<PaymentMethod> paymentMethods;
```

```
const Customer({
```

```
  required this.id,
```

```
  required this.name,
```

```
  required this.email,
```

```
  required this.phone,
```

```
  required this.address,
```

```

    this.companyName,
    this.abn,
    this.preferredContact,
    this.language,
    this.receiveNotifications = true,
    this.receiveNewsletters = false,
    required this.createdDate,
    this.lastLoginDate,
    this.accountStatus = 'active',
    this.defaultPaymentMethodId,
    this.paymentMethods = const [],
  });
}

```

```

class PaymentMethod {
  final String id;
  final String type; // card, bank_transfer, paypal, cash
  final String? maskedNumber;
  final DateTime? expiryDate;
  final bool isDefault;

  const PaymentMethod({
    required this.id,
    required this.type,
    this.maskedNumber,
    this.expiryDate,
    this.isDefault = false,
  });
}

```

// Customer service request (what they booked)

```

class ServiceRequest {
  final String id;
  final String customerId;
  final String serviceType;
  final String description;
  final DateTime requestedDate;
  final DateTime? scheduledDate;
  final Location serviceLocation;

  final ServiceRequestStatus status;
  final double estimatedPrice;
  final String? quoteId;
  final String? jobId;
}

```

```

// History
final DateTime createdAt;
final List<ServiceRequestUpdate> updates;

const ServiceRequest({
  required this.id,
  required this.customerId,
  required this.serviceType,
  required this.description,
  required this.requestedDate,
  this.scheduledDate,
  required this.serviceLocation,
  required this.status,
  required this.estimatedPrice,
  this.quoteId,
  this.jobId,
  required this.createdAt,
  required this.updates,
});
}

enum ServiceRequestStatus {
  requested,
  quote_sent,
  quote_accepted,
  scheduled,
  in_progress,
  completed,
  quote_rejected,
  cancelled,
}

class ServiceRequestUpdate {
  final String id;
  final DateTime timestamp;
  final String type; // status_change, message, quote, invoice, etc
  final String message;
  final String? data;

  const ServiceRequestUpdate({
    required this.id,
    required this.timestamp,
    required this.type,

```

```
        required this.message,  
        this.data,  
    });  
}
```

```
// Customer portal controller
```

```
class CustomerPortalController extends GetxController {  
    final customerRepository = Get.find<CustomerRepository>();  
    final serviceRepository = Get.find<ServiceRepository>();
```

```
  
    final currentCustomer = Rxn<Customer>();  
    final serviceRequests = <ServiceRequest>[].obs;  
    final pastServices = <ServiceRequest>[].obs;  
    final invoices = <Invoice>[].obs;  
    final quotes = <Quote>[].obs;
```

```
  
    @override  
    void onInit() {  
        super.onInit();  
        loadCustomerData();  
    }
```

```
  
    Future<void> loadCustomerData() async {  
        final customerId = getCurrentCustomerId();
```

```
  
        // Get customer info  
        final customer = await customerRepository.getCustomer(customerId);  
        currentCustomer.value = customer;
```

```
  
        // Get active service requests  
        final active = await serviceRepository.getActiveRequests(customerId);  
        serviceRequests.value = active;
```

```
  
        // Get past services  
        final past = await serviceRepository.getPastRequests(customerId);  
        pastServices.value = past;
```

```
  
        // Get invoices  
        final inv = await serviceRepository.getInvoices(customerId);  
        invoices.value = inv;
```

```
  
        // Get quotes  
        final q = await serviceRepository.getQuotes(customerId);  
        quotes.value = q;
```

```
}
```

```
Future<void> requestService({
    required String serviceType,
    required String description,
    required DateTime preferredDate,
    required Location location,
}) async {
    try {
        final request = ServiceRequest(
            id: generateId(),
            customerId: getCurrentCustomerId(),
            serviceType: serviceType,
            description: description,
            requestedDate: DateTime.now(),
            requestedDate: preferredDate,
            serviceLocation: location,
            status: ServiceRequestStatus.requested,
            estimatedPrice: 0,
            createdAt: DateTime.now(),
            updates: [
                ServiceRequestUpdate(
                    id: generateId(),
                    timestamp: DateTime.now(),
                    type: 'created',
                    message: 'Service request created',
                ),
            ],
        );

        await serviceRepository.saveRequest(request);
        serviceRequests.add(request);

        // Notify business
        Get.find<NotificationService>().notifyNewServiceRequest(request);

        Get.snackbar('Success', 'Service request submitted');
    } catch (e) {
        Get.snackbar('Error', 'Failed to submit request');
    }
}
```

```
Future<void> acceptQuote(String quoteId) async {
    final quote = quotes.firstWhere((q) => q.id == quoteId);
```

```

final request = serviceRequests.firstWhere((r) => r.quoteId == quoteId);

request.status = ServiceRequestStatus.quote_accepted;

// Update service request
await serviceRepository.updateRequest(request);

// Notify business that quote was accepted
Get.find<NotificationService>().notifyQuoteAccepted(quoteId);

Get.snackbar('Success', 'Quote accepted. We will schedule your service soon.');
```

```

}

Future<void> rejectQuote(String quoteId, String reason) async {
  final request = serviceRequests
    .firstWhere((r) => r.quoteId == quoteId);

  request.status = ServiceRequestStatus.quote_rejected;
  request.updates.add(ServiceRequestUpdate(
    id: generateId(),
    timestamp: DateTime.now(),
    type: 'quote_rejected',
    message: 'Quote rejected: $reason',
  ));

  await serviceRepository.updateRequest(request);

  Get.snackbar('Done', 'Quote rejected');
```

```

}
}

// Database schema for customer portal
void _createCustomerPortalTables(Database db, int version) async {
  await db.execute("""
    CREATE TABLE customers (
      id TEXT PRIMARY KEY,
      name TEXT NOT NULL,
      email TEXT UNIQUE NOT NULL,
      phone TEXT,
      address TEXT,
      company_name TEXT,
      abn TEXT,
      preferred_contact TEXT,
      receive_notifications INTEGER DEFAULT 1,

```

```
    created_at TEXT NOT NULL,  
    last_login TEXT,  
    account_status TEXT DEFAULT 'active'  
);  
"";
```

```
await db.execute("""  
CREATE TABLE service_requests (  
    id TEXT PRIMARY KEY,  
    customer_id TEXT NOT NULL,  
    service_type TEXT NOT NULL,  
    description TEXT NOT NULL,  
    requested_date TEXT NOT NULL,  
    scheduled_date TEXT,  
    location_latitude REAL,  
    location_longitude REAL,  
    location_address TEXT,  
    status TEXT NOT NULL,  
    estimated_price REAL,  
    quote_id TEXT,  
    job_id TEXT,  
    created_at TEXT NOT NULL,  
    FOREIGN KEY(customer_id) REFERENCES customers(id)  
);  
"";
```

```
await db.execute("""  
CREATE TABLE service_updates (  
    id TEXT PRIMARY KEY,  
    request_id TEXT NOT NULL,  
    timestamp TEXT NOT NULL,  
    type TEXT NOT NULL,  
    message TEXT NOT NULL,  
    data TEXT,  
    FOREIGN KEY(request_id) REFERENCES service_requests(id)  
);  
"";
```

```
// Create indices  
await db.execute('CREATE INDEX idx_requests_customer ON  
service_requests(customer_id)');  
await db.execute('CREATE INDEX idx_requests_status ON service_requests(status)');  
await db.execute('CREATE INDEX idx_updates_request ON service_updates(request_id)');  
}
```

PHASE 1: BOOKING & SERVICE REQUEST (Weeks 3-5)

Service booking, quote requests, preferred dates, location selection

```
class BookingScreen extends StatefulWidget {
  @override
  _BookingScreenState createState() => _BookingScreenState();
}

class _BookingScreenState extends State<BookingScreen> {
  final controller = Get.put(CustomerPortalController());

  final serviceTypeController = TextEditingController();
  final descriptionController = TextEditingController();
  final addressController = TextEditingController();

  DateTime selectedDate = DateTime.now();
  Location? selectedLocation;
  String? selectedServiceType;

  final _formKey = GlobalKey<FormState>();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Book Service')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: EdgeInsets.all(16),
          children: [
            // Service type
            DropdownFormField(
              label: 'Service Type',
              items: ['Plumbing', 'Electrical', 'HVAC', 'General Maintenance'],
              onChanged: (value) => setState(() => selectedServiceType = value),
            ),
            SizedBox(height: 16),

            // Description
            TextFormField(
              controller: descriptionController,
              decoration: InputDecoration(
```



```

        label: Text('What do you need?'),
        hintText: 'Describe the service or problem',
      ),
      maxLines: 4,
      validator: (value) => value?.isEmpty ?? true ? 'Required' : null,
    ),
    SizedBox(height: 16),

    // Preferred date
    ListTile(
      title: Text('Preferred Date'),
      subtitle: Text(selectedDate.toString().split(' ')[0]),
      onTap: () => _selectDate(),
    ),
    SizedBox(height: 16),

    // Address
    TextFormField(
      controller: addressController,
      decoration: InputDecoration(label: Text('Service Location')),
      onTap: () => _selectLocation(),
      validator: (value) => value?.isEmpty ?? true ? 'Required' : null,
    ),
    SizedBox(height: 16),

    // Current address suggestion
    if (selectedLocation != null)
      Chip(
        label: Text(selectedLocation!.address),
        onDelete: () => setState(() => selectedLocation = null),
      ),

    SizedBox(height: 24),

    // Submit
    ElevatedButton(
      onPressed: _submitRequest,
      child: Text('REQUEST SERVICE'),
    ),
  ],
),
);
}

```

```

void _selectDate() {
  showDatePicker(
    context: context,
    initialDate: selectedDate,
    firstDate: DateTime.now(),
    lastDate: DateTime.now().add(Duration(days: 90)),
  ).then((date) {
    if (date != null) {
      setState(() => selectedDate = date);
    }
  });
}

```

```

void _selectLocation() async {
  // Use maps to select location
  // Could integrate Google Places API

  // For now, use current location
  final location = await Get.find<LocationService>().getCurrentLocation();
  if (location != null) {
    setState(() {
      selectedLocation = location;
      addressController.text = location.address;
    });
  }
}

```

```

void _submitRequest() {
  if (!_formKey.currentState!.validate()) return;

  controller.requestService(
    serviceType: selectedServiceType ?? "",
    description: descriptionController.text,
    preferredDate: selectedDate,
    location: selectedLocation ?? Location(
      latitude: 0,
      longitude: 0,
      address: addressController.text,
    ),
  );
}

```

@override

```

void dispose() {
  serviceTypeController.dispose();
  descriptionController.dispose();
  addressController.dispose();
  super.dispose();
}
}

```

PHASE 2: TRACKING & STATUS UPDATES (Weeks 6-8)

Real-time service request tracking, maps, ETA, serviceman details

```

class ServiceTrackingScreen extends StatelessWidget {
  final String requestId = Get.arguments as String;
  final controller = Get.put(CustomerPortalController());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Service Status')),
      body: Obx(() {
        final request = controller.serviceRequests
          .firstWhereOrNull((r) => r.id == requestId);

        if (request == null) {
          return Center(child: Text('Service not found'));
        }

        return ListView(
          padding: EdgeInsets.all(16),
          children: [
            // Status badge
            Card(
              color: _statusColor(request.status),
              child: Padding(
                padding: EdgeInsets.all(16),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      _statusText(request.status),
                      style: TextStyle(
                        color: Colors.white,
                        fontSize: 18,

```

```

        fontWeight: FontWeight.bold,
      ),
    ),
    SizedBox(height: 8),
    if (request.status == ServiceRequestStatus.in_progress)
      Text(
        'Your serviceman is on the way',
        style: TextStyle(color: Colors.white),
      ),
    if (request.status == ServiceRequestStatus.scheduled)
      Text(
        'Scheduled for ${request.scheduledDate.toString().split(' ')[0]}',
        style: TextStyle(color: Colors.white),
      ),
  ],
),
),
),
),
),

```

```

SizedBox(height: 16),

```

```

// Map (if in progress)
if (request.status == ServiceRequestStatus.in_progress)
  Card(
    child: SizedBox(
      height: 300,
      child: GoogleMap(
        initialCameraPosition: CameraPosition(
          target: LatLng(
            request.serviceLocation.latitude,
            request.serviceLocation.longitude,
          ),
          zoom: 15,
        ),
      ),
    ),
  ),
),
),
),
),

```

```

SizedBox(height: 16),

```

```

// Service details
Card(
  child: Padding(
    padding: EdgeInsets.all(16),
  ),
),

```

```
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('Service Details', style: TextStyle(fontWeight: FontWeight.bold)),
    SizedBox(height: 12),
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text('Type'),
        Text(request.serviceType),
      ],
    ),
    SizedBox(height: 8),
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text('Location'),
        Expanded(
          child: Text(
            request.serviceLocation.address,
            textAlign: TextAlign.right,
            overflow: TextOverflow.ellipsis,
          ),
        ),
      ],
    ),
    SizedBox(height: 8),
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text('Scheduled'),
        Text(request.scheduledDate?.toString().split(' ')[0] ?? 'Not yet'),
      ],
    ),
  ],
),
SizedBox(height: 16),

// Timeline
Card(
  child: Padding(
```

```
padding: EdgeInsets.all(16),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('Timeline', style: TextStyle(fontWeight: FontWeight.bold)),
    SizedBox(height: 12),
    ...request.updates.map((update) => Padding(
      padding: EdgeInsets.only(bottom: 12),
      child: Row(
        children: [
          Icon(
            _updateIcon(update.type),
            size: 20,
            color: Colors.grey,
          ),
          SizedBox(width: 12),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  update.message,
                  style: TextStyle(fontWeight: FontWeight.bold),
                ),
                Text(
                  update.timestamp.toString().split('.')[0],
                  style: TextStyle(fontSize: 12, color: Colors.grey),
                ),
              ],
            ),
          ),
        ],
      ),
    ),
  ],
),
SizedBox(height: 16),

// Contact button
ElevatedButton.icon(
  onPressed: () => _contactBusiness(),
```

```

        icon: Icon(Icons.phone),
        label: Text('Contact Us'),
    ),
],
);
}),
);
}

```

```

String _statusText(ServiceRequestStatus status) {
    return {
        ServiceRequestStatus.requested: 'Request Submitted',
        ServiceRequestStatus.quote_sent: 'Quote Available',
        ServiceRequestStatus.quote_accepted: 'Confirmed',
        ServiceRequestStatus.scheduled: 'Scheduled',
        ServiceRequestStatus.in_progress: 'In Progress',
        ServiceRequestStatus.completed: 'Completed',
        ServiceRequestStatus.quote_rejected: 'Quote Rejected',
        ServiceRequestStatus.cancelled: 'Cancelled',
    }[status] ?? 'Unknown';
}

```

```

Color _statusColor(ServiceRequestStatus status) {
    return {
        ServiceRequestStatus.requested: Colors.blue,
        ServiceRequestStatus.quote_sent: Colors.orange,
        ServiceRequestStatus.quote_accepted: Colors.green,
        ServiceRequestStatus.scheduled: Colors.teal,
        ServiceRequestStatus.in_progress: Colors.purple,
        ServiceRequestStatus.completed: Colors.greenAccent,
        ServiceRequestStatus.quote_rejected: Colors.red,
        ServiceRequestStatus.cancelled: Colors.grey,
    }[status] ?? Colors.grey;
}

```

```

IconData _updateIcon(String type) {
    return {
        'created': Icons.add_circle,
        'status_change': Icons.update,
        'message': Icons.message,
        'quote': Icons.description,
        'scheduled': Icons.calendar_today,
    }[type] ?? Icons.info;
}

```

```

void _contactBusiness() {
  Get.dialog(
    AlertDialog(
      title: Text('Contact Us'),
      content: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          ListTile(
            title: Text('Call'),
            trailing: Icon(Icons.call),
            onTap: () => launchUrl(Uri.parse('tel:1300123456')),
          ),
          ListTile(
            title: Text('Text'),
            trailing: Icon(Icons.message),
            onTap: () => launchUrl(Uri.parse('sms:0412345678')),
          ),
          ListTile(
            title: Text('Chat'),
            trailing: Icon(Icons.chat),
            onTap: () => Navigator.pop(Get.context!),
          ),
        ],
      ),
    ),
  );
}

```

PHASE 3: QUOTES & INVOICING (Weeks 9-11)

View quotes, accept/reject, invoices, payment options

```

class InvoicePaymentScreen extends StatelessWidget {
  final String invoiceId = Get.arguments as String;
  final controller = Get.put(CustomerPortalController());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Invoice')),
      body: Obx(() {
        final invoice = controller.invoices

```



```

        .firstWhereOrNull((inv) => inv.id == invoiceId);

if (invoice == null) {
    return Center(child: Text('Invoice not found'));
}

return ListView(
    padding: EdgeInsets.all(16),
    children: [
        // Invoice header
        Card(
            child: Padding(
                padding: EdgeInsets.all(16),
                child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                        Text(
                            'Invoice #${invoice.id.substring(0, 8)}',
                            style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                        ),
                        SizedBox(height: 8),
                        Text('Issued: ${invoice.issueDate.toString().split(' ')[0]}'),
                        Text('Due: ${invoice.dueDate.toString().split(' ')[0]}'),
                    ],
                ),
            ),
        ),
        SizedBox(height: 16),

        // Line items
        Card(
            child: Padding(
                padding: EdgeInsets.all(16),
                child: Column(
                    children: [
                        ...invoice.lineItems.map((item) => Padding(
                            padding: EdgeInsets.only(bottom: 12),
                            child: Row(
                                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                                children: [
                                    Expanded(
                                        child: Text(item.description),
                                    ),
                                ],
                            ),
                        ),
                    ],
                ),
            ),
        ),
    ],
);

```



```

        ElevatedButton(
          onPressed: () => _initiatePayment(invoice),
          child: Text('PAY NOW'),
        ),

        SizedBox(height: 16),

        // Download invoice
        OutlinedButton.icon(
          onPressed: () => _downloadInvoice(invoice),
          icon: Icon(Icons.download),
          label: Text('Download PDF'),
        ),
      ],
    );
  },
);
}

void _initiatePayment(Invoice invoice) {
  Get.dialog(
    AlertDialog(
      title: Text('Payment Methods'),
      content: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          ListTile(
            title: Text('Credit Card'),
            onTap: () => _paymentFlow(invoice, 'card'),
          ),
          ListTile(
            title: Text('Bank Transfer'),
            onTap: () => _paymentFlow(invoice, 'bank'),
          ),
          ListTile(
            title: Text('PayPal'),
            onTap: () => _paymentFlow(invoice, 'paypal'),
          ),
        ],
      ),
    ),
  );
}

```

```

void _paymentFlow(Invoice invoice, String method) {
  // Redirect to payment processor (user's API)
  Get.snackbar('Processing', 'Redirecting to payment...');
  // In production: integrate with Stripe, Square, etc
}

void _downloadInvoice(Invoice invoice) {
  Get.snackbar('Downloading', 'Invoice PDF');
  // Generate PDF and download
}
}

```

PHASE 4: SUPPORT & COMMUNICATION (Weeks 12-14)

Live chat, FAQ, support tickets, push notifications

```

class SupportScreen extends StatefulWidget {
  @override
  _SupportScreenState createState() => _SupportScreenState();
}

```

```

class _SupportScreenState extends State<SupportScreen> {
  final messageController = TextEditingController();
  final messages = <Message>[].obs;

```

```

  @override
  void initState() {
    super.initState();
    _loadMessages();
    _setupRealtimeUpdates();
  }

```

```

  void _loadMessages() {
    // Load chat history
  }

```

```

  void _setupRealtimeUpdates() {
    // Listen for new messages via Signal Core or WebSocket
  }

```

```

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Support')),

```

```

body: Column(
  children: [
    // FAQ section
    Padding(
      padding: EdgeInsets.all(16),
      child: TextField(
        decoration: InputDecoration(
          hintText: 'Search FAQ...',
          prefixIcon: Icon(Icons.search),
          border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(8),
          ),
        ),
      ),
    ),
  ),
),

Expanded(
  child: DefaultTabController(
    length: 2,
    child: Column(
      children: [
        TabBar(
          tabs: [
            Tab(text: 'FAQ'),
            Tab(text: 'Chat Support'),
          ],
        ),
        Expanded(
          child: TabBarView(
            children: [
              // FAQ list
              ListView(
                children: [
                  FAQItem(
                    question: 'How long until my service?',
                    answer: 'Most services scheduled within 3-5 business days.',
                  ),
                  FAQItem(
                    question: 'Can I reschedule?',
                    answer: 'Yes, you can reschedule up to 24 hours before.',
                  ),
                  FAQItem(
                    question: 'What payment methods?',
                    answer: 'We accept card, bank transfer, PayPal, and cash.',
                  ),
                ],
              ),
            ],
          ),
        ),
      ],
    ),
  ),
),

```

```

    ),
  ],
),

// Chat
Column(
  children: [
    Expanded(
      child: Obx(() => ListView(
        reverse: true,
        children: messages.map((msg) => Padding(
          padding: EdgeInsets.all(8),
          child: Align(
            alignment: msg.isFromCustomer
              ? Alignment.centerRight
              : Alignment.centerLeft,
            child: Container(
              constraints: BoxConstraints(
                maxWidth: MediaQuery.of(context).size.width * 0.7,
              ),
              decoration: BoxDecoration(
                color: msg.isFromCustomer
                  ? Colors.blue
                  : Colors.grey[300],
                borderRadius: BorderRadius.circular(12),
              ),
              padding: EdgeInsets.all(12),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    msg.text,
                    style: TextStyle(
                      color: msg.isFromCustomer
                        ? Colors.white
                        : Colors.black,
                    ),
                  ),
                  Text(
                    msg.timestamp.toString().split('.')[0],
                    style: TextStyle(
                      fontSize: 10,
                      color: msg.isFromCustomer
                        ? Colors.white70

```

[illegible]

```

    );
}

void _sendMessage() {
    final text = messageController.text;
    if (text.isEmpty) return;

    final message = Message(
        id: generateId(),
        text: text,
        isFromCustomer: true,
        timestamp: DateTime.now(),
    );

    messages.add(message);
    messageController.clear();

    // Send to support team (via API)
}

@override
void dispose() {
    messageController.dispose();
    super.dispose();
}
}

class FAQItem extends StatefulWidget {
    final String question;
    final String answer;

    const FAQItem({required this.question, required this.answer});

    @override
    _FAQItemState createState() => _FAQItemState();
}

class _FAQItemState extends State<FAQItem> {
    bool expanded = false;

    @override
    Widget build(BuildContext context) {
        return Card(
            margin: EdgeInsets.symmetric(horizontal: 16, vertical: 8),

```



```

child: ExpansionTile(
  title: Text(widget.question, style: TextStyle(fontWeight: FontWeight.bold)),
  children: [
    Padding(
      padding: EdgeInsets.all(16),
      child: Text(widget.answer),
    ),
  ],
),
);
}
}

```

```

class Message {
  final String id;
  final String text;
  final bool isFromCustomer;
  final DateTime timestamp;

  const Message({
    required this.id,
    required this.text,
    required this.isFromCustomer,
    required this.timestamp,
  });
}

```

PHASE 5: ACCOUNT & PREFERENCES (Weeks 15-16)
 Customer profile, payment methods, preferences, history

```

class AccountScreen extends StatelessWidget {
  final controller = Get.put(CustomerPortalController());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('My Account')),
      body: Obx(() {
        final customer = controller.currentCustomer.value;

        if (customer == null) {
          return Center(child: CircularProgressIndicator());
        }
      })
    );
  }
}

```

```
return ListView(
padding: EdgeInsets.all(16),
children: [
// Profile section
Card(
child: Padding(
padding: EdgeInsets.all(16),
child: Column(
crossAxisAlignment: CrossAxisAlignment.start,
children: [
Text('Profile', style: TextStyle(fontWeight: FontWeight.bold, fontSize: 16)),
 SizedBox(height: 16),
ListTile(
title: Text('Name'),
subtitle: Text(customer.name),
),
ListTile(
title: Text('Email'),
subtitle: Text(customer.email),
),
ListTile(
title: Text('Phone'),
subtitle: Text(customer.phone),
),
ListTile(
title: Text('Address'),
subtitle: Text(customer.address),
),
SizedBox(height: 16),
ElevatedButton(
onPressed: () => _editProfile(customer),
child: Text('Edit Profile'),
),
],
),
),
),
),
SizedBox(height: 16),
// Payment methods
Card(
child: Padding(
```

```

padding: EdgeInsets.all(16),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('Payment Methods', style: TextStyle(fontWeight: FontWeight.bold)),
    SizedBox(height: 12),
    ...customer.paymentMethods.map((method) => Padding(
      padding: EdgeInsets.only(bottom: 8),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text('${method.type}: ${method.maskedNumber}'),
          if (method.isDefault) Chip(label: Text('Default')),
        ],
      ),
    )),
    SizedBox(height: 8),
    ElevatedButton.icon(
      onPressed: () => _addPaymentMethod(),
      icon: Icon(Icons.add),
      label: Text('Add Payment Method'),
    ),
  ],
),
),
),
),
),

```

```

SizedBox(height: 16),

```

```

// Preferences

```

```

Card(
  child: Padding(
    padding: EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text('Preferences', style: TextStyle(fontWeight: FontWeight.bold)),
        SizedBox(height: 12),
        SwitchListTile(
          title: Text('Receive Notifications'),
          value: customer.receiveNotifications,
          onChanged: (value) {
            // Update preference
          },

```

```

    ),
    SwitchListTile(
      title: Text('Receive Newsletter'),
      value: customer.receiveNewsletters,
      onChanged: (value) {
        // Update preference
      },
    ),
  ],
),
),
),
),

    SizedBox(height: 16),

    // Service history
    Card(
      child: Padding(
        padding: EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('Service History', style: TextStyle(fontWeight: FontWeight.bold)),
            SizedBox(height: 12),
            Text('Total Services: ${controller.pastServices.length}'),
            SizedBox(height: 8),
            ElevatedButton(
              onPressed: () => Get.toNamed('/history'),
              child: Text('View History'),
            ),
          ],
        ),
      ),
    ),
    ),
    ),

    SizedBox(height: 24),

    // Logout
    OutlinedButton(
      onPressed: () => _logout(),
      child: Text('Log Out'),
    ),
  ],
);

```

```

    }},
  );
}

void _editProfile(Customer customer) {
  // Open edit profile dialog
}

void _addPaymentMethod() {
  // Open add payment method dialog
}

void _logout() {
  Get.find<AuthService>().logout();
  Get.offAllNamed('/login');
}
}

```

FINAL STATISTICS - TITAN PORTAL

TOTAL TIME: 16 weeks (4 months)

TEAM: 2 devs + 1 designer + 1 QA

CODE: 18,000-22,000 LOC | 200+ tests

FEATURES:

- ✓ Service booking (request, dates, location)
- ✓ Quote viewing & acceptance
- ✓ Real-time service tracking (maps)
- ✓ Invoicing & payment
- ✓ Live chat support
- ✓ FAQ database
- ✓ Account management
- ✓ Payment method management
- ✓ Service history
- ✓ Push notifications
- ✓ Offline capability
- ✓ Mobile + Web (PWA)

SECURITY:

- └─ AES-256 encryption
- └─ TLS 1.3
- └─ Token auth

└ Audit logging

INTEGRATIONS:

- └ Maps (Google Maps)
- └ Payment (Stripe/Square/PayPal - user's API)
- └ Chat (WebSocket or Signal Core)
- └ Email (transactional)
- └ Notifications (Push)

PLATFORMS:

- └ iOS 14.0+
- └ Android 8.0+
- └ Web (PWA - responsive)

PERFORMANCE:

- └ Startup: < 2s
- └ Actions: < 300ms
- └ Maps: < 500ms
- └ Battery: Minimal

COMPLETE TITANBOS SUITE SUMMARY

Four Core Apps:

1. Titan Command - Dispatch, invoicing, payroll (22 weeks, 28k-32k LOC)
2. Titan Go - Field operations, tracking, voice (20 weeks, 25k-30k LOC)
3. Titan Trust - Compliance, assets, fleet, contracts (22 weeks, 28k-32k LOC)
4. Titan Portal - Customer-facing booking & tracking (16 weeks, 18k-22k LOC)

Total Development: ~80-90 weeks (18-20 months) | ~100k-115k LOC | 800+ tests
Team: 10-12 people total across all apps
Market Ready: Production-grade, NDIS/disability compliant, Australian workplace standards

 Enterprise-grade business operating system for service industries.